PORTFOLIO

SLMAN AL-ODANI

NHL STENDEN EMMEN
Computer Science
Student Number : 5638364
Number of EC's : 240
Slman.al-odani@student.nhlstenden.com
2025-2029

Introduction:

As a Computer Science student, this portfolio reflects the journey I've taken to explore, build, and understand the digital world. From foundational programming concepts to hands-on projects, each section showcases not just what I've learned, but how I've applied it. Whether it's solving problems through code, analyzing systems, or experimenting with new technologies, I've aimed to approach each challenge with curiosity and persistence. This collection is a snapshot of my growth and a starting point for where I hope to go next.

My GitHub name is slmanalodani.

Table of Contents

Introduction: 1
Computer science assignments: 3
Deep thinking assignment (week 4.1): 4
Week 4.2 Assignments: 4
Week 4.3 Assignments: 9
Week 4.4 + 4.5 Assignments: 24
Week 4.6 Assignments: 43
Professional skills: 49

Computer science assignments:

Deep thinking assignment (week 4.1):

Interview 3 people about their experiences with AI.
Write a short analysis of identifying current challenges in AI:

After talking with three people from different ages and backgrounds, a few clear challenges in today's AI systems start to stand out.

My mother's experience highlights a very common issue: AI sometimes struggles to understand what the user wants, especially when the user phrases things in a more natural or less technical way. She mentioned that the AI "doesn't get what she wants" even when she feels she's asking clearly. Part of this might be the way older generations communicate with technology — they expect it to understand context the same way a human would. When it doesn't, it feels frustrating and unintuitive. This shows that AI still has a long way to go in adapting to different communication styles and levels of digital familiarity.

My sister pointed out another major challenge: AI hallucinations. She noticed that sometimes the AI confidently gives her links or information that simply doesn't exist. This is a known limitation — AI models can generate answers that *sound* right but aren't actually grounded in real data. For users who rely on AI for research or quick information, this creates confusion and reduces trust. It's a reminder that accuracy and reliability are still inconsistent.

My brother's experience adds a third challenge: AI's limited memory in conversations. He mentioned that the AI doesn't keep up with the flow of their talk, forcing him to repeat things or "refresh its memory." This reflects a real limitation in many AI systems: they can only hold a certain amount of context at once, and they don't truly "remember" past conversations unless specifically designed to. For users who want long, continuous discussions, this can feel like talking to someone who keeps forgetting what was just said.

Taken together, these three perspectives show that while AI is powerful, it still struggles with understanding, accuracy, and long-term context. These challenges affect different people in different ways, but they all point to the same conclusion: AI is improving fast, yet it still needs to become more intuitive, more reliable, and more human-aware to fully meet people's expectations.

Week 4.2 Assignments:

1) Deep thinking assignment
   Formulate a "How Might We" question.
   Create two fictional user profiles with their challenges and needs

Fictional User Profile 1 — “Sara, the Overwhelmed First-Year Student”

Basic Info

- **Age:** 19
- **Background:** First-year international student
- **Tech Skill Level:** Beginner
- **Environment:** Lives in a shared dorm, noisy, limited study space

Challenges

- **Struggles to understand digital platforms** used by the university (email, submission portals).
- **Feels embarrassed asking for help**, so she often tries alone and gets stuck.
- **Overwhelmed by notifications**, deadlines, and multiple apps.
- **English is not her first language**, so instructions feel confusing.

Needs

- A **simple, guided interface** that explains steps clearly.
- **Context-aware help** that adapts to her skill level.
- A **centralized dashboard** showing deadlines, tasks, and instructions in one place.
- Tools that **reduce cognitive load**, like templates, reminders, and auto-organization.

Fictional User Profile 2 — “Jack, the Busy Working Student”

Basic Info

- **Age:** 27
- **Background:** Part-time student, full-time retail worker
- **Tech Skill Level:** Intermediate
- **Environment:** Studies late at night after work

Challenges

- **Time-poor** — juggling shifts, classes, and assignments.
- **Often forgets deadlines** because his schedule changes weekly.
- **Switches between devices** (phone at work, laptop at home), causing lost progress.
- **Feels frustrated** when tools are slow or require too many steps.

Needs

- **Fast, mobile-friendly tools** that sync across devices.
- **Smart reminders** that adapt to his changing schedule.
- **Automation** for repetitive tasks (formatting, citations, file organization).
- A system that **saves progress instantly** and works offline when needed.

2) GIRank

Assignment description:
You get a family tree in which each individual passes on genetic influence to their offspring.
The goal is to calculate a "Genetic Influence Rank" (GIRank) for each ancestor. This figure indicates how much influence that person has on the current generation.

1. Just as PageRank calculates the influence of a web page through links, you calculate how much genetic influence an ancestor has through multiple generations.
2. In contrast to a simple 50/50 split of genes per parent, individuals with many offspring must spread their influence.
3. Some ancestors can have indirect influence through multiple generations.

Game Rules & Details:
1. Input: You get a list of ancestors and their direct children, for example:

```
family_tree = {
"Alice": ["Bob", "Charlie"],
"Bob": ["David"],
"Charlie": ["Eve", "Frank"],
"David": ["George"],
"Eve": ["Hannah"],
    "Frank": ["Isaac"]
    }
```

1. Each parent divides their genetic influence equally among their children.
2. Children pass on this influence to their own offspring.
3. Parents with more children spread their influence over a larger group, just like PageRank does.
4. Calculate the "Genetic Influence Rank" (GIRank) with an iterative algorithm

The code:

```python
def girank(family_tree, iterations=50):
    # Collect all unique people
    people = set(family_tree.keys())
    for children in family_tree.values():
        people.update(children)

    # Initialize influence equally
    N = len(people)
    rank = {p: 1 / N for p in people}

    # Build parent → children mapping
    children_of = {p: family_tree.get(p, []) for p in people}

    # Build child → parents mapping
    parents_of = {p: [] for p in people}
    for parent, children in family_tree.items():
        for child in children:
            parents_of[child].append(parent)

    # Step 1: Spread influence downward
    for _ in range(iterations):
        new_rank = {p: 0 for p in people}

        for parent, children in children_of.items():
            if len(children) == 0:
                new_rank[parent] += rank[parent]
            else:
                share = rank[parent] / len(children)
                for child in children:
                    new_rank[child] += share

        rank = new_rank

    # Step 2: Ancestors get the influence of their descendants
    ancestor_score = {p: 0 for p in people}

    for person, influence in rank.items():
        current = person
        while parents_of[current]:
            for parent in parents_of[current]:
                ancestor_score[parent] += influence
            current = parents_of[current][0]  # move upward

    return ancestor_score
```

```python
# ------------------------
# Family tree
# ------------------------

family_tree = {
    "Alice": ["Bob", "Charlie"],
    "Bob": ["David"],
    "Charlie": ["Eve", "Frank"],
    "David": ["George"],
    "Eve": ["Hannah"],
    "Frank": ["Isaac"]
}

scores = girank(family_tree)

# Print sorted results
sorted_scores = sorted(scores.items(), key=lambda x: x[1], reverse=True)

for person, score in sorted_scores:
    print(f"{person}: {score:.6f}")
```

The result:

```
GIRANK/ $ python girank.py
Alice: 1.000000
Charlie: 0.611111
Bob: 0.388889
David: 0.388889
Frank: 0.305556
Eve: 0.305556
Isaac: 0.000000
Hannah: 0.000000
George: 0.000000
```

Short reflection:

Working on the GIRank algorithm helped me understand how influence can spread through a network, similar to how PageRank works on the web. Instead of webpages and links, I applied the same idea to a family tree, where parents pass influence to their children. One thing I learned is that influence becomes weaker when it is divided among

many children, and stronger when a branch is narrow. This explains why ancestors with fewer children can sometimes have more influence than those with many.

I also learned how iterative algorithms work: the influence values change step by step until they stabilize. Implementing this in Python helped me practice working with dictionaries, loops, and graph-like structures. Finally, seeing the results made the concept clearer — for example, Charlie has more influence than Bob because Charlie has more descendant paths leading to the final generation.

Week 4.3 Assignments:

Crossword:

In this assignment, you have to design and implement an AI system capable of automatically solving a crossword puzzle. The task requires you to model the crossword as a constraint satisfaction problem (CSP), where each empty slot in the grid becomes a variable, and every possible word becomes part of its domain. You must enforce node consistency by filtering out words that don't match the required lengths and apply arc consistency (AC-3) to ensure that intersecting words share compatible letters.

You also have to implement a backtracking search algorithm that tries to assign words to all slots while respecting every constraint. To make the solver efficient, the assignment requires using heuristics such as Minimum Remaining Values, Degree heuristic, and Least Constraining Value. By the end, the AI should be able to take a crossword structure and a word list and generate a complete, valid crossword solution.

The Code:

```
import sys


from crossword import *




class CrosswordCreator():


    def __init__(self, crossword):
        """
        Create new CSP crossword generate.
        """
        self.crossword = crossword
        self.domains = {
            var: self.crossword.words.copy()
            for var in self.crossword.variables
        }


    def letter_grid(self, assignment):
        """
        Return 2D array representing a given assignment.
        """
        letters = [
            [None for _ in range(self.crossword.width)]
            for _ in range(self.crossword.height)
        ]
```

```
        for variable, word in assignment.items():
            direction = variable.direction
            for k in range(len(word)):
                i = variable.i + (k if direction == Variable.DOWN else 0)
                j = variable.j + (k if direction == Variable.ACROSS else 0)
                letters[i][j] = word[k]
        return letters


    def print(self, assignment):
        """
        Print crossword assignment to the terminal.
        """
        letters = self.letter_grid(assignment)
        for i in range(self.crossword.height):
            for j in range(self.crossword.width):
                if self.crossword.structure[i][j]:
                    print(letters[i][j] or " ", end="")
                else:
                    print("", end="")
            print()


    def save(self, assignment, filename):
        """
        Save crossword assignment to an image file.
        """
        from PIL import Image, ImageDraw, ImageFont
        cell_size = 100
        cell_border = 2
        interior_size = cell_size - 2 * cell_border
        letters = self.letter_grid(assignment)
```

```
        # Create a blank canvas
        img = Image.new(
            "RGBA",
            (self.crossword.width * cell_size,
             self.crossword.height * cell_size),
            "black"
        )
        font = ImageFont.truetype("assets/fonts/OpenSans-Regular.ttf", 80)
        draw = ImageDraw.Draw(img)

        for i in range(self.crossword.height):
            for j in range(self.crossword.width):

                rect = [
                    (j * cell_size + cell_border,
                     i * cell_size + cell_border),
                    ((j + 1) * cell_size - cell_border,
                     (i + 1) * cell_size - cell_border)
                ]
                if self.crossword.structure[i][j]:
                    draw.rectangle(rect, fill="white")
                    if letters[i][j]:
                        _, _, w, h = draw.textbbox((0, 0), letters[i][j],
font=font)
                        draw.text(
                            (rect[0][0] + ((interior_size - w) / 2),
                             rect[0][1] + ((interior_size - h) / 2) - 10),
                            letters[i][j], fill="black", font=font
                        )
```

```
        img.save(filename)


    def solve(self):
        """
        Enforce node and arc consistency, and then solve the CSP.
        """
        self.enforce_node_consistency()
        self.ac3()
        return self.backtrack(dict())


    def enforce_node_consistency(self):
        """
        Update `self.domains` such that each variable is node-consistent.
        (Remove any values that are inconsistent with a variable's unary
         constraints; in this case, the length of the word.)
        """
        # loop through the variables
        for v in self.crossword.variables:
            # loop through a copy of the domain for that variable
            for word in self.domains[v].copy():
                #check if the length of the word is fit for the variables
length
                if len(word) != v.length:
                # remove word from the domian for that variaable
                    self.domains[v].remove(word)


    def revise(self, x, y):
        """
        Make variable `x` arc consistent with variable `y`.
        To do so, remove values from `self.domains[x]` for which there is no
        possible corresponding value for `y` in `self.domains[y]`.
```

```
        Return True if a revision was made to the domain of `x`; return
        False if no revision was made.
        """
        revised = False
        i , j = self.crossword.overlaps[x, y]
        # loop through the variable x domain
        for word_x in self.domains[x].copy():
            supported = False
            # do the same for variable y
            for word_y in self.domains[y]:
                # compare the words
                if word_x[i] == word_y[j]:
                    supported = True
                    break # to do the next word for y
            if not supported:
                # remove the word_x
                self.domains[x].remove(word_x)
                revised = True
        return revised


    def ac3(self, arcs=None):
        """
        Update `self.domains` such that each variable is arc consistent.
        If `arcs` is None, begin with initial list of all arcs in the problem.
        Otherwise, use `arcs` as the initial list of arcs to make consistent.

        Return True if arc consistency is enforced and no domains are empty;
        return False if one or more domains end up empty.
        """
```

```
        queue = []


        if arcs is None:
            # build initial queue of ALL arcs
            # loop through all the variables
            for x in self.crossword.variables:
                for y in self.crossword.variables:
                    #not the same variable
                    if x != y and self.crossword.overlaps[x, y] is not None:
                        queue.append((x,y))


        else:
            # use arcs as the queue
            queue = list(arcs)


        # call revise if there are queue
        while queue:
            # pop x , y at the 0 index
            (x , y) = queue.pop(0)
            if self.revise(x,y) == True:
                # check if the domain empty
                if len(self.domains[x]) == 0:
                    return False


                ol_neighbors = self.crossword.neighbors(x)
                for nieghbor in ol_neighbors:
                    if nieghbor != y:
                        # add nieghbor to the queue
                        queue.append((nieghbor, x))
        return True
```

```
    def assignment_complete(self, assignment):
        """
        Return True if `assignment` is complete (i.e., assigns a value to each
        crossword variable); return False otherwise.
        """
        # An assignment is complete if every crossword variable is assigned to
a value
        for v in self.crossword.variables:
            if v not in assignment :
                return False
        return True


    def consistent(self, assignment, var=None, value=None):


        # If var and value are provided → check only that pair
        if var is not None and value is not None:


            # length check
            if len(value) != var.length:
                return False


            # duplicate check
            for other in assignment:
                if assignment[other] == value:
                    return False


            # overlap check
```

```
            for neighbor in self.crossword.neighbors(var):
                if neighbor in assignment:
                    i, j = self.crossword.overlaps[var, neighbor]
                    if value[i] != assignment[neighbor][j]:
                        return False

            return True

        # Otherwise → check entire assignment (check50 mode)
        for v in assignment:
            word = assignment[v]

            # length check
            if len(word) != v.length:
                return False

            # duplicate check
            for other in assignment:
                if other != v and assignment[other] == word:
                    return False

            # overlap check
            for neighbor in self.crossword.neighbors(v):
                if neighbor in assignment:
                    i, j = self.crossword.overlaps[v, neighbor]
                    if word[i] != assignment[neighbor][j]:
                        return False

        return True
```

```
def order_domain_values(self, var, assignment):
    """
    Return a list of values in the domain of `var`, in order by
    the number of values they rule out for neighboring variables.
    The first value in the list, for example, should be the one
    that rules out the fewest values among the neighbors of `var`.
    """
    # we neeed a list of words
    words =[]
    # loop through all the variables
    for word in self.domains[var]:
        # counter for elemination
        elm_counter = 0
        for neighbor in self.crossword.neighbors(var):
            if neighbor not in assignment:
                (i, j) = self.crossword.overlaps[var, neighbor]
                if (i, j) is None:
                    continue
                for neighbor_word in self.domains[neighbor]:
                    if word[i] != neighbor_word [j]:
                        elm_counter +=1
        words.append((word ,elm_counter))
    # sort by elm
    words.sort(key=lambda x: x[1])
    # return only the words:
    return [w for w, _ in words]



def select_unassigned_variable(self, assignment):
```

```
        """
        Return an unassigned variable not already part of `assignment`.
        Choose the variable with the minimum number of remaining values
        in its domain. If there is a tie, choose the variable with the highest
        degree. If there is a tie, any of the tied variables are acceptable
        return values.
        """
        #list of unassigned variables
        un_var = []
        # loop through the variables
        for v in self.crossword.variables:
            if v not in assignment:
                un_var.append(v)
        # how many words in the domain for that variable compute MRV
        min_value = min(len(self.domains[v]) for v in un_var)
        # This filters all variables that match that minimum size.
        candidates = [v for v in un_var if len(self.domains[v]) == min_value]
        # Degree heuristic — break ties
        best = max(candidates, key=lambda v: len(self.crossword.neighbors(v)))
        # return best_candidate
        return best



    def backtrack(self, assignment):
        """
        Using Backtracking Search, take as input a partial assignment for the
        crossword and return a complete assignment if possible to do so.

        `assignment` is a mapping from variables (keys) to words (values).
```

```
        If no assignment is possible, return None.
        """
        # check if the assignment complete
        if self.assignment_complete(assignment):
            return assignment
        # select the unassigned variable
        var = self.select_unassigned_variable(assignment)
        # loop through the values of that varibale
        for value in self.order_domain_values(var, assignment):
            # chcek consistency
            if self.consistent(assignment, var, value):
                # assign
                assignment[var] = value
                # recurse
                result = self.backtrack(assignment)
                if result is not None:
                    return result
                # undo
                del assignment[var]
        # failure
        return None


def main():

    # Check usage
    if len(sys.argv) not in [3, 4]:
        sys.exit("Usage: python generate.py structure words [output]")

    # Parse command-line arguments
    structure = sys.argv[1]
```

```
    words = sys.argv[2]
    output = sys.argv[3] if len(sys.argv) == 4 else None

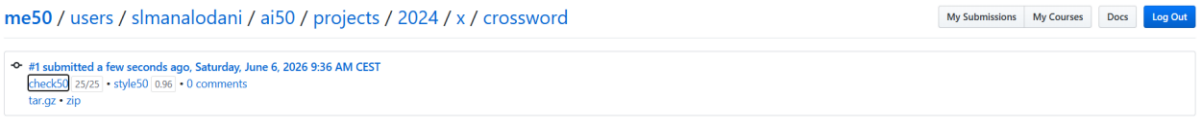
    # Generate crossword
    crossword = Crossword(structure, words)
    creator = CrosswordCreator(crossword)
    assignment = creator.solve()

    # Print result
    if assignment is None:
        print("No solution.")
    else:
        creator.print(assignment)
        if output:
            creator.save(assignment, output)


if __name__ == "__main__":
    main()
```

Picture from the terminal:





https://submit.cs50.io/check50/0ab583890f7f0b1d07fd6a242de40543fea0ce29

2) Design Thinking Assignment

Brainstorm 10 AI solutions for the defined problem.
Select the 3 best ideas and describe why they have potential.

10 AI Solutions for Habit Lab:

- **AI Habit Predictor** — Predicts which strategy (A or B) is more likely to work for the user before the experiment starts, based on past behavior.
- **Smart Metric Recommender** — Suggests the best metric to measure the habit (e.g., wake-up time, steps, screen time).
- **AI Experiment Designer** — Helps the user write a clear experiment question and choose strong A/B strategies.
- **Daily Insight Generator** — Gives short AI-generated insights based on the user's daily logs.
- **Anomaly Detector** — Detects unusual days (e.g., sickness, travel) and prevents them from ruining the experiment.
- **Motivation Nudger** — Sends personalized motivational messages based on the user's progress.

- **AI Habit Coach** — Gives recommendations on how to improve the next experiment after finishing one.
- **Pattern Finder** — Finds hidden patterns in the user's data (e.g., "Option B works better on weekends").
- **Adaptive Experiment Duration** — Uses AI to decide if the experiment needs more days to be statistically meaningful.
- **Result Summarizer** — Generates a clean, simple explanation of which option won and why.

Top 3 Ideas:

1) AI Habit Predictor

Why it has potential:

- Helps the user choose better A/B strategies before starting
- Reduces wasted time on experiments that won't work
- Makes the app feel smart and personalized
- Increases user trust because the system "understands" their behavior

2) AI Experiment Designer

Why it has potential:

- Many users don't know how to write a good experiment question
- AI can suggest strong Option A and Option B strategies
- Ensures the experiment is scientifically valid
- Makes the app easier for beginners

3) Pattern Finder

Why it has potential:

- Users often don't see patterns in their own behavior
- AI can detect trends like:
    - "Option A works better on weekdays"
    - "Option B works better when you sleep early"

- Makes the results more meaningful
- Helps users design better future experiments

Week 4.4 + 4.5 Assignments:

1) Shopping Assignment:

In this assignment, you have to design and implement an AI system capable of predicting whether a user will complete an online purchase based on their browsing behavior. The task requires you to treat the dataset as a **supervised learning problem**, where each row represents a user session and the goal is to classify it as "purchase" or "no purchase."

You must preprocess the dataset by converting categorical features into numerical values, handling Boolean fields, and preparing the data for training. The assignment requires you to split the dataset into **training and testing sets**, then train a **k-nearest neighbors (k-NN) classifier** to learn patterns from the data. The model predicts a label by examining the most similar past sessions and using majority voting.

To evaluate performance, you must compute **accuracy**, **sensitivity**, and **specificity**, which measure how well the model identifies true buyers and non-buyers. By the end, the AI should be able to take a new user session and automatically predict whether the user is likely to make a purchase, demonstrating how supervised learning can be applied to real-world e-commerce data.

The Code:

```
import csv
import sys


from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier


TEST_SIZE = 0.4




def main():
```

```
    # Check command-line arguments
    if len(sys.argv) != 2:
        sys.exit("Usage: python shopping.py data")

    # Load data from spreadsheet and split into train and test sets
    evidence, labels = load_data(sys.argv[1])
    X_train, X_test, y_train, y_test = train_test_split(
        evidence, labels, test_size=TEST_SIZE
    )

    # Train model and make predictions
    model = train_model(X_train, y_train)
    predictions = model.predict(X_test)
    sensitivity, specificity = evaluate(y_test, predictions)

    # Print results
    print(f"Correct: {(y_test == predictions).sum()}")
    print(f"Incorrect: {(y_test != predictions).sum()}")
    print(f"True Positive Rate: {100 * sensitivity:.2f}%")
    print(f"True Negative Rate: {100 * specificity:.2f}%")


def load_data(filename):
    """
    Load shopping data from a CSV file `filename` and convert into a list of
    evidence lists and a list of labels. Return a tuple (evidence, labels).

    evidence should be a list of lists, where each list contains the
    following values, in order:
```

```
        - Administrative, an integer
        - Administrative_Duration, a floating point number
        - Informational, an integer
        - Informational_Duration, a floating point number
        - ProductRelated, an integer
        - ProductRelated_Duration, a floating point number
        - BounceRates, a floating point number
        - ExitRates, a floating point number
        - PageValues, a floating point number
        - SpecialDay, a floating point number
        - Month, an index from 0 (January) to 11 (December)
        - OperatingSystems, an integer
        - Browser, an integer
        - Region, an integer
        - TrafficType, an integer
        - VisitorType, an integer 0 (not returning) or 1 (returning)
        - Weekend, an integer 0 (if false) or 1 (if true)

    labels should be the corresponding list of labels, where each label
    is 1 if Revenue is true, and 0 otherwise.
    """
    # months dic
    months = {"Jan": 0, "Feb": 1, "Mar": 2,
        "Apr": 3, "May": 4, "June": 5,
        "Jul": 6, "Aug": 7, "Sep": 8,
        "Oct": 9, "Nov": 10, "Dec": 11,}

    # open the csv file
    with open(filename) as file:
```

```
        # load it into a csv reader
        reader = csv.reader(file)


        evidence= []
        labels = []


        #skip the header
        next(reader)


        for row in reader:
            current = []


            current.append(int(row[0]))
            current.append(float(row[1]))
            current.append(int(row[2]))
            current.append(float(row[3]))
            current.append(int(row[4]))
            current.append(float(row[5]))
            current.append(float(row[6]))
            current.append(float(row[7]))
            current.append(float(row[8]))
            current.append(float(row[9]))
            current.append(months[row[10]])
            current.append(int(row[11]))
            current.append(int(row[12]))
            current.append(int(row[13]))
            current.append(int(row[14]))
            visitor_value = 1 if row[15] == "Returning_Visitor" else 0
            current.append(visitor_value)
            weekend_value = 0 if row[16] == "FALSE" else 1
```

```
            current.append(weekend_value)
            evidence.append(current)
            label_value = 1 if row[17] == "TRUE" else 0
            labels.append(label_value)

        return evidence, labels


def train_model(evidence, labels):
    """
    Given a list of evidence lists and a list of labels, return a
    fitted k-nearest neighbor model (k=1) trained on the data.
    """
    model =  KNeighborsClassifier(n_neighbors=1) # create model // k = 1
    model.fit(evidence, labels) # train model
    return model # give it back


def evaluate(labels, predictions):
    """
    Given a list of actual labels and a list of predicted labels,
    return a tuple (sensitivity, specificity).

    Assume each label is either a 1 (positive) or 0 (negative).

    `sensitivity` should be a floating-point value from 0 to 1
    representing the "true positive rate": the proportion of
    actual positive labels that were accurately identified.

    `specificity` should be a floating-point value from 0 to 1
    representing the "true negative rate": the proportion of
    actual negative labels that were accurately identified.
```

```
    """
    # Counters for all four possible outcomes
    true_positive = 0      # actual = 1, predicted = 1
    false_negative = 0     # actual = 1, predicted = 0
    true_negative = 0      # actual = 0, predicted = 0
    false_positive = 0     # actual = 0, predicted = 1

    # Loop through actual labels and predicted labels at the same time
    for actual, predicted in zip(labels, predictions):

        # If the real label is positive (1)
        if actual == 1:

            # And the model also predicted positive → true positive
            if predicted == 1:
                true_positive += 1

            # Model predicted negative → false negative
            else:
                false_negative += 1

        # If the real label is negative (0)
        else:

            # And the model predicted negative → true negative
            if predicted == 0:
                true_negative += 1

            # Model predicted positive → false positive
            else:
```

```
                false_positive += 1


    # Sensitivity = how many actual positives we correctly predicted
    sensitivity = true_positive / (true_positive + false_negative)


    # Specificity = how many actual negatives we correctly predicted
    specificity = true_negative / (true_negative + false_positive)

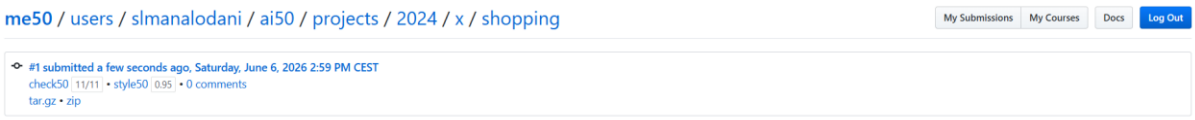

    # Return both values as a tuple
    return sensitivity, specificity



if __name__ == "__main__":
    main()
```

Picture from the terminal:

```
Shopping/shopping_folder/shopping/ $  python shopping.py shopping.csv
Correct: 4061
Incorrect: 871
True Positive Rate: 36.89%
True Negative Rate: 90.26%
Shopping/shopping_folder/shopping/ $
```



https://submit.cs50.io/check50/4c2033cb5e4d1e39d9ddb3f53962b795b487ffea

2) Nim Assignment:

In this assignment, you have to design and implement an AI agent capable of learning to play the game of Nim optimally through reinforcement learning. Instead of exploring the entire game tree with Minimax, the task requires you to use **Q-learning**, a model-free learning algorithm in which the agent improves by playing repeated games against itself.

Each configuration of piles represents a **state**, and each possible removal of objects represents an **action**. The AI maintains a **Q-table** that maps `(state, action)` pairs to numerical values estimating how good each move is. After every move, the agent updates its Q-values using the formula:

$$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \max_{a'} Q(s',a') - Q(s,a))$$

To balance exploration and exploitation, the agent uses an **epsilon-greedy policy**: with probability ε it chooses a random action, and with probability 1−ε it chooses the action with the highest Q-value. Over many self-play games, the Q-values converge toward optimal play.

By the end of the assignment, your AI should be able to consistently choose strong moves based on its learned Q-values, demonstrating how reinforcement learning can be applied to create intelligent game-playing agents.

The Code:

```python
import math
import random
import time



class Nim():


    def __init__(self, initial=[1, 3, 5, 7]):
        """
        Initialize game board.
        Each game board has
            - `piles`: a list of how many elements remain in each pile
            - `player`: 0 or 1 to indicate which player's turn
            - `winner`: None, 0, or 1 to indicate who the winner is
        """
        self.piles = initial.copy()
        self.player = 0
        self.winner = None


    @classmethod
    def available_actions(cls, piles):
        """
        Nim.available_actions(piles) takes a `piles` list as input
        and returns all of the available actions `(i, j)` in that state.


        Action `(i, j)` represents the action of removing `j` items
        from pile `i` (where piles are 0-indexed).
        """
        actions = set()
        for i, pile in enumerate(piles):
```

```
            for j in range(1, pile + 1):
                actions.add((i, j))
        return actions


    @classmethod
    def other_player(cls, player):
        """
        Nim.other_player(player) returns the player that is not
        `player`. Assumes `player` is either 0 or 1.
        """
        return 0 if player == 1 else 1


    def switch_player(self):
        """
        Switch the current player to the other player.
        """
        self.player = Nim.other_player(self.player)


    def move(self, action):
        """
        Make the move `action` for the current player.
        `action` must be a tuple `(i, j)`.
        """
        pile, count = action


        # Check for errors
        if self.winner is not None:
            raise Exception("Game already won")
        elif pile < 0 or pile >= len(self.piles):
            raise Exception("Invalid pile")
```

```
        elif count < 1 or count > self.piles[pile]:
            raise Exception("Invalid number of objects")

        # Update pile
        self.piles[pile] -= count
        self.switch_player()

        # Check for a winner
        if all(pile == 0 for pile in self.piles):
            self.winner = self.player


class NimAI():

    def __init__(self, alpha=0.5, epsilon=0.1):
        """
        Initialize AI with an empty Q-learning dictionary,
        an alpha (learning) rate, and an epsilon rate.

        The Q-learning dictionary maps `(state, action)`
        pairs to a Q-value (a number).
         - `state` is a tuple of remaining piles, e.g. (1, 1, 4, 4)
         - `action` is a tuple `(i, j)` for an action
        """
        self.q = dict()
        self.alpha = alpha
        self.epsilon = epsilon

    def update(self, old_state, action, new_state, reward):
        """
```

```
        Update Q-learning model, given an old state, an action taken
        in that state, a new resulting state, and the reward received
        from taking that action.
        """
        old = self.get_q_value(old_state, action)
        best_future = self.best_future_reward(new_state)
        self.update_q_value(old_state, action, old, reward, best_future)

    def get_q_value(self, state, action):
        """
        Return the Q-value for the state `state` and the action `action`.
        If no Q-value exists yet in `self.q`, return 0.
        """
        state_tuple = tuple(state)
        if (state_tuple, action) in self.q:
            q_value = self.q[state_tuple, action]
            return q_value
        else:
            return 0



    def update_q_value(self, state, action, old_q, reward, future_rewards):
        """
        Update the Q-value for the state `state` and the action `action`
        given the previous Q-value `old_q`, a current reward `reward`,
        and an estiamte of future rewards `future_rewards`.

        Use the formula:


        Q(s, a) <- old value estimate
```

```
                + alpha * (new value estimate - old value estimate)

    where `old value estimate` is the previous Q-value,
    `alpha` is the learning rate, and `new value estimate`
    is the sum of the current reward and estimated future rewards.
    """
    # convert state into tuple
    state_tuple = tuple(state)
    # get the old q
    old_q = self.get_q_value(state,action)
    # new q
    new_q = old_q + self.alpha * (reward + future_rewards - old_q)
    # store it
    self.q[(state_tuple, action)] = new_q



def best_future_reward(self, state):
    """
    Given a state `state`, consider all possible `(state, action)`
    pairs available in that state and return the maximum of all
    of their Q-values.

    Use 0 as the Q-value if a `(state, action)` pair has no
    Q-value in `self.q`. If there are no available actions in
    `state`, return 0.
    """
    # get all the actions
    actions = Nim.available_actions(state)
    maxq_value = -math.inf
    if actions:
```

```
            for a in actions:
                q_value = self.get_q_value(state, a)
                if q_value >= maxq_value:
                    maxq_value = q_value
            return maxq_value
        return 0




    def choose_action(self, state, epsilon=True):
        """
        Given a state `state`, return an action `(i, j)` to take.

        If `epsilon` is `False`, then return the best action
        available in the state (the one with the highest Q-value,
        using 0 for pairs that have no Q-values).

        If `epsilon` is `True`, then with probability
        `self.epsilon` choose a random available action,
        otherwise choose the best action available.

        If multiple actions have the same Q-value, any of those
        options is an acceptable return value.
        """
        actions = Nim.available_actions(state)

        # If epsilon=False → no exploration allowed
        if not epsilon:
            return max(actions, key=lambda a: self.get_q_value(state, a))
```

```
        # Otherwise use epsilon-greedy with self.epsilon
        if random.random() < self.epsilon:
            return random.choice(list(actions))
        else:
            return max(actions, key=lambda a: self.get_q_value(state, a))




def train(n):
    """
    Train an AI by playing `n` games against itself.
    """

    player = NimAI()

    # Play n games
    for i in range(n):
        print(f"Playing training game {i + 1}")
        game = Nim()

        # Keep track of last move made by either player
        last = {
            0: {"state": None, "action": None},
            1: {"state": None, "action": None}
        }

        # Game loop
        while True:

            # Keep track of current state and action
```

```
            state = game.piles.copy()
            action = player.choose_action(game.piles)

            # Keep track of last state and action
            last[game.player]["state"] = state
            last[game.player]["action"] = action

            # Make move
            game.move(action)
            new_state = game.piles.copy()

            # When game is over, update Q values with rewards
            if game.winner is not None:
                player.update(state, action, new_state, -1)
                player.update(
                    last[game.player]["state"],
                    last[game.player]["action"],
                    new_state,
                    1
                )
                break

            # If game is continuing, no rewards yet
            elif last[game.player]["state"] is not None:
                player.update(
                    last[game.player]["state"],
                    last[game.player]["action"],
                    new_state,
                    0
                )
```

```
    print("Done training")

    # Return the trained AI
    return player


def play(ai, human_player=None):
    """
    Play human game against the AI.
    `human_player` can be set to 0 or 1 to specify whether
    human player moves first or second.
    """

    # If no player order set, choose human's order randomly
    if human_player is None:
        human_player = random.randint(0, 1)

    # Create new game
    game = Nim()

    # Game loop
    while True:

        # Print contents of piles
        print()
        print("Piles:")
        for i, pile in enumerate(game.piles):
            print(f"Pile {i}: {pile}")
        print()
```

```
        # Compute available actions
        available_actions = Nim.available_actions(game.piles)
        time.sleep(1)

        # Let human make a move
        if game.player == human_player:
            print("Your Turn")
            while True:
                pile = int(input("Choose Pile: "))
                count = int(input("Choose Count: "))
                if (pile, count) in available_actions:
                    break
                print("Invalid move, try again.")

        # Have AI make a move
        else:
            print("AI's Turn")
            pile, count = ai.choose_action(game.piles, epsilon=False)
            print(f"AI chose to take {count} from pile {pile}.")

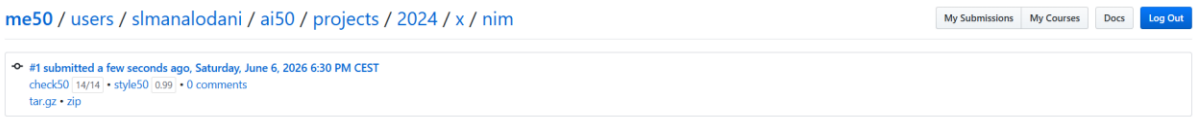
        # Make move
        game.move((pile, count))

        # Check for winner
        if game.winner is not None:
            print()
            print("GAME OVER")
            winner = "Human" if game.winner == human_player else "AI"
            print(f"Winner is {winner}")
```

```
            return
```

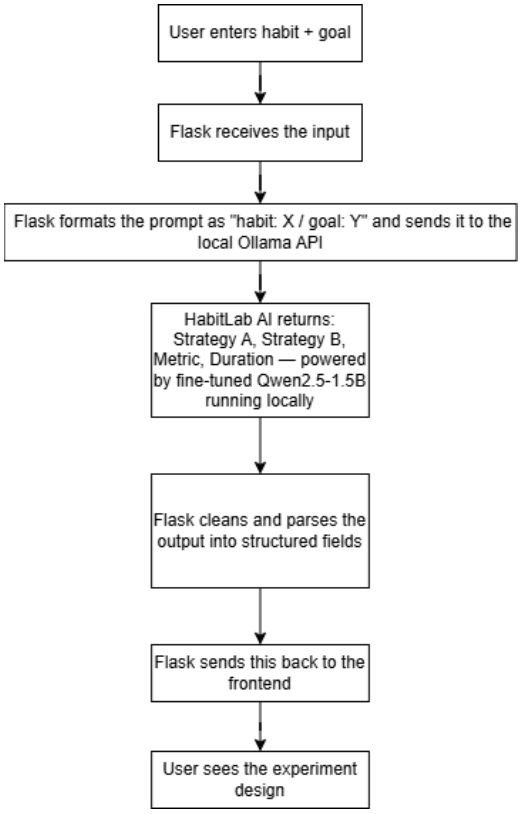
Pic from the terminal:





https://submit.cs50.io/check50/59b1e5b94dbc560e3cd5cd9d4ccef56c7e417405

3) **Design Thinking Assignment:**
   Create a low-fidelity prototype: AI flow schema, wireframe, or a basic Python script.


User enters habit + goal
Flask receives the input
Flask formats the prompt as "habit: X / goal: Y" and sends it to the local Ollama API
HabitLab AI returns: Strategy A, Strategy B, Metric, Duration — powered by fine-tuned Qwen2.5-1.5B running locally
Flask cleans and parses the output into structured fields
Flask sends this back to the frontend
User sees the experiment design

Week 4.6 Assignments:

1) Traffic:

   In this assignment, you have to design and implement an AI agent that learns to control a traffic intersection using reinforcement learning. Each combination of waiting cars across four directions represents a state, and choosing which direction gets the green light represents an action. The AI maintains a Q-table mapping (state, action) pairs to values that estimate how good each decision is, and updates those values after every phase using the formula $Q(s,a) \leftarrow Q(s,a) + \alpha(r + \max_{a'} Q(s',a') - Q(s,a))$, where r is the number of cars that cleared the intersection. To balance trying new actions and sticking with what works, the agent uses an epsilon-greedy policy: with probability ε it picks randomly, and with probability 1−ε it picks the best known action. Over many simulated episodes, the agent learns to consistently prioritize the more congested direction, demonstrating how reinforcement learning can be applied to traffic management.

   The code:

```python
import cv2
import numpy as np
import os
import sys
import tensorflow as tf

from sklearn.model_selection import train_test_split

EPOCHS = 10
IMG_WIDTH = 30
IMG_HEIGHT = 30
NUM_CATEGORIES = 43
TEST_SIZE = 0.4



def main():
```

```
    # Check command-line arguments
    if len(sys.argv) not in [2, 3]:
        sys.exit("Usage: python traffic.py data_directory [model.h5]")

    # Get image arrays and labels for all image files
    images, labels = load_data(sys.argv[1])

    # Split data into training and testing sets
    labels = tf.keras.utils.to_categorical(labels)
    x_train, x_test, y_train, y_test = train_test_split(
        np.array(images), np.array(labels), test_size=TEST_SIZE
    )

    # Get a compiled neural network
    model = get_model()

    # Fit model on training data
    model.fit(x_train, y_train, epochs=EPOCHS)

    # Evaluate neural network performance
    model.evaluate(x_test,  y_test, verbose=2)

    # Save model to file
    if len(sys.argv) == 3:
        filename = sys.argv[2]
        model.save(filename)
        print(f"Model saved to {filename}.")



def load_data(data_dir):
```

```
images = []
labels = []

# Loop through each category folder (0 to NUM_CATEGORIES - 1)
for category in range(NUM_CATEGORIES):
    category_path = os.path.join(data_dir, str(category))

    # Skip if the folder doesn't exist
    if not os.path.isdir(category_path):
        continue

    # Walk through the folder and any subdirectories inside it
    for root, dirs, files in os.walk(category_path):
        for filename in files:
            filepath = os.path.join(root, filename)

            # Read the image as a numpy array
            img = cv2.imread(filepath)

            # Skip if OpenCV couldn't read the file
            if img is None:
                continue

            # Resize to required dimensions
            img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))

            images.append(img)
            labels.append(category)
```

```
    return (images, labels)


def get_model():


    model = tf.keras.models.Sequential([


        # Explicit input layer — the recommended way in newer Keras versions
        tf.keras.Input(shape=(IMG_WIDTH, IMG_HEIGHT, 3)),


        # First convolutional layer: learns basic features like edges and
colors
        tf.keras.layers.Conv2D(32, (3, 3), activation="relu"),


        # Reduces image size by taking the max value in each 2x2 region
        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),


        # Second convolutional layer: learns more complex features
        tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),


        # Another pooling layer to further reduce size
        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),


        # Flatten 2D feature maps into a 1D vector for the dense layers
        tf.keras.layers.Flatten(),


        # Fully connected hidden layer
        tf.keras.layers.Dense(128, activation="relu"),


        # Randomly disables 50% of neurons during training to prevent
overfitting
```

```
        tf.keras.layers.Dropout(0.5),

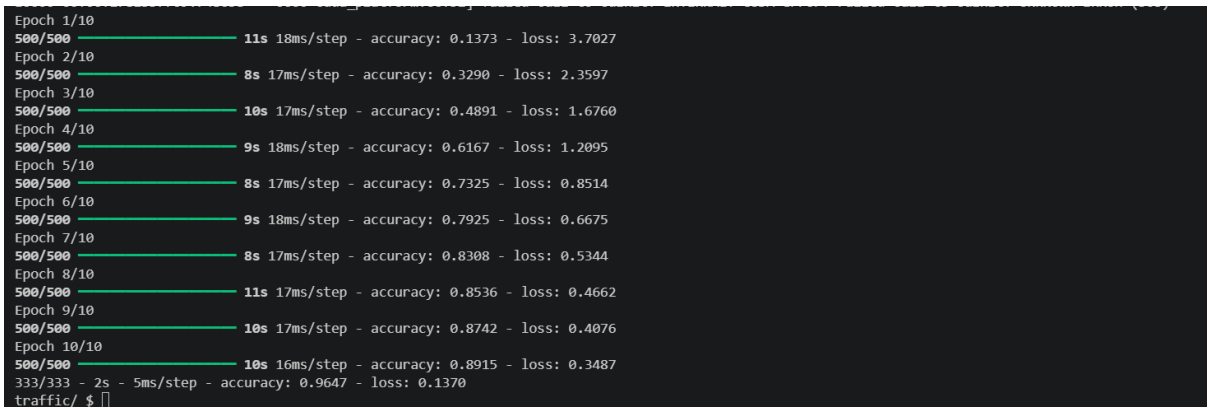

        # Output layer: one neuron per category
        tf.keras.layers.Dense(NUM_CATEGORIES, activation="softmax")
    ])

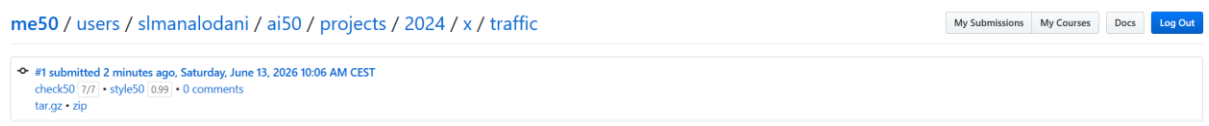

    model.compile(
        optimizer="adam",
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )


    return model


if __name__ == "__main__":
    main()
```

Image from the terminal:

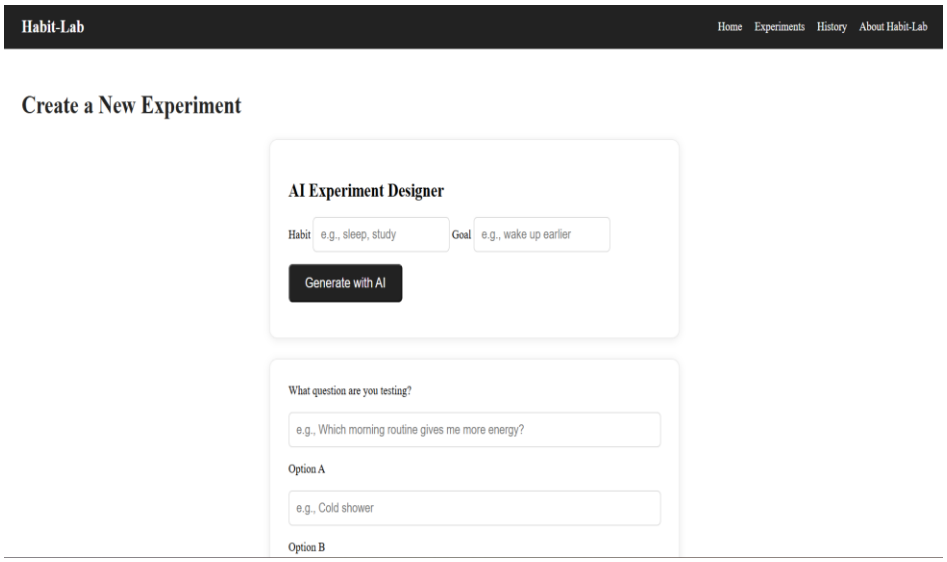




https://submit.cs50.io/check50/b0e26c2d56f9114c3149d549d7490ddaf08e6558

2) **Design Thinking Assignment:**

Work out your prototype in code (e.g. a chatbot, image recognition or recommender system).
Use an AI framework such as OpenAI API, TensorFlow, or Scikit-learn:
The codes in GitHub: https://github.com/slmanalodani/solo_innovate
Pic of the AI:



Professional skills:

Period 3 Assignments:

Week1 Setting & Achieving Goals: SMART(ER):

Setting & Achieving Goals: SMART(ER)
Assignment Week 1, Period 1 (02-02/06-02)

Name: Slman Al-Odani
Date: 8/2/2026

1. Download this document and answer the questions here under
2. When finished, upload your assignment on Blackboard, in your Portfolio
3. **Deadline: 08-02, at 23:59**

Do you think -after this theory class- that setting SMART goals actually helps in reaching important matters? Honesty is appreciated; naturally you will not be graded on a defiant answer

Your answer: I think having a method that helps me organize and reach my goals is really useful. As long as I have the motivation, a structure like SMART goals can definitely support me in staying focused and moving forward

Describe a situation where you set a goal for yourself and took actual steps to reach this goals. Elaborate please. Just realize: you all have done this at least once in your life

Your answer: A situation where I set a goal for myself was when I decided to get my Dutch B1 diploma. I had about a year and a half to prepare, and my goal was to pass it before the new academic year started so I could enroll in Emmen. I knew it was important, so I committed to studying consistently. I practiced every day, followed my teachers' advice, and pushed myself even when it felt difficult. In the end, all that effort paid off — I managed to get the diploma just in time.

Per category describe a goal you would like to reach.

Short-term goal: To finish the measurement report for last period project.
Long-term goal: To graduate from university.
Personal goal: To get my driving license.
Professional goal: To be more confident in coding.
Relational goal: To improve the way I communicate with other people.

Now, pick one goal that is really important to reach. Describe this goal SMARTER

The goal: To graduate from university.
S pecific: I want to successfully complete all the required courses, projects, and exams in my program so I can earn my university degree.
M easurable: I can measure my progress by passing each semester, completing assignments on time, and keeping my grades at the required level.

Acceptable: This goal is important to me because graduating will open opportunities for my future, and I'm willing to put in the work needed to achieve it.
Realistic: It's realistic because I have the ability, support, and resources to study, ask for help when needed, and stay consistent throughout the program.
Time-bound: My target is to graduate by the end of my program's official duration
Evaluate: I will regularly check my progress each semester, look at my grades, and adjust my study habits if something isn't working.
Result/Reward: The reward will be earning my degree, improving my career opportunities, and feeling proud that I completed something important for my future.

Week 4: Prioritizing Using the Time Management Matrix:

Prioritizing Using the Time Management Matrix
Assignment Week 4, Period 4 (02-03/06-03)

Name : Slman Al-Odani
Date : 8/3/2026

1. Download this document and answer the questions here under
2. When finished, upload your assignment on Blackboard, in your Portfolio
3. **Deadline: 08-03, at 23:59**

In order to prioritise, you need to know what your values are. After all, your values determine your prioritization. So this is very personal. For more information on values see 'Additional text in this week's PPT. Here are 10 lined up. Grade each one of them with either:

- **Very applicable to me**
- **Applicable to me**
- **Not applicable to me**
- **Not applicable to me at all**

Ambition: Applicable to me
Reliability: Very applicable to me
Expertise: Applicable to me
Discipline: Applicable to me
Honesty: Very applicable to me
Health: Applicable to me
Independence: Very applicable to me
Quality: Applicable to me
Personal Growth: Very applicable to me
Respect: Very applicable to me

Perhaps you're not aware of this, but you have (subconsciously) prioritized in the past. If necessary dig deep and describe a situation where you had to prioritize on important matters. What (values) determined your choices?

Your answer:

A situation where I had to prioritize was during a busy study period when I had multiple deadlines at the same time. I had to choose between preparing for an exam, finishing a group assignment, and helping a friend who needed support. What determined my choices were my values of **reliability**, **respect**, and **personal growth**. I focused first on the tasks that had the biggest impact on my academic progress, because personal growth and ambition matter to me. At the same time, I made sure to communicate clearly with my group and my friend, because reliability and respect are important values in how I treat others.

Looking back: did you subconsciously use the Time Management Matrix in some form? Elaborate

Your answer:
Looking back, I think I subconsciously used the Time Management Matrix. I focused first on the tasks that were both important and urgent, like the exam preparation and the assignment deadline. Then I planned the important but less urgent things, such as reviewing extra material. I postponed or reduced time spent on non-important activities, like social media or casual conversations. Even though I didn't think about the matrix directly, I naturally separated tasks based on urgency and importance.

Now, make a to-do list for this period. Then list them Quadrant-wise. So make a Matrix with the 4 quadrants and upload this along with this document. Remember, the time Management Matrix is a personal thing, determined by your values. Therefore, wrong or stupid answers (quadrants) do not exist. Good luck

Q1 Important – Urgent activities:
Q2 Important – Non urgent activities:
Q3 Non Important – Urgent activities:
Q4 Non Important – Non Urgent activities:

To-Do List for This Period:

- Finish Week 4 assignment.
- Prepare for upcoming exam
- Attend classes
- Review lecture notes
- Clean room
- Grocery shopping
- Check social media
- Watch a series
- Exercise
- Plan next week's schedule

Time Management Matrix

Q1 – Important & Urgent

- Finish Week 4 assignment
- Prepare for upcoming exam
- Attend classes

Q2 – Important & Not Urgent

- Review lecture notes
- Exercise
- Plan next week's schedule
- Personal growth activities (reading, skill development)

Q3 – Not Important & Urgent

- Grocery shopping
- Responding quickly to non-important messages

Q4 – Not Important & Not Urgent

- Checking social media
- Watching a series
- Gaming or other leisure activities

Week6 Feedback:

Feedback
Assignment Week 6, Period 3 (16-03/20-03)

Name Slman Alodani
Date

1. Download this document, make the assignments here under and answer the questions
2. When finished, upload your assignment on Blackboard, in your Portfolio
3. **Deadline: 22-03, at 23:59**

Go to this week's power point to look up some information.

1. **Write an essay of minimum 500 words about the importance of feedback, including at least the following topics:**
    - **Why it's important to be able to give proper feedback? Involve the 21st century skills also in your essay**
    - **Why feedback is different from criticizing**
    - **Mention the ground rules for proper feedback. Do not include the BECD-method in your answer, since this is an instrument for proper feedback, not a rule**
    - **Describe an example of effective feedback you received in the (recent) past**

Your essay:

Feedback plays a crucial role in personal growth, professional development, and effective communication. In today's rapidly changing world, the ability to give and receive proper feedback has become more important than ever. As we navigate the demands of the 21st century—where collaboration, critical thinking, communication, and adaptability are essential—feedback acts as a bridge between where we are and where we want to be. It helps individuals understand their strengths, identify areas for improvement, and adjust their behavior or performance in meaningful ways.

One of the main reasons proper feedback is so important is because it directly supports several key 21st-century skills. For example, **communication** is strengthened when people learn how to express observations clearly and respectfully. **Collaboration** improves when team members feel safe sharing constructive insights with one another. **Critical thinking** is encouraged when feedback helps someone reflect on their choices and consider alternative approaches. Even **creativity** benefits from feedback, because it allows people to refine their ideas and push them further. In a world where teamwork and innovation are essential, the ability to give proper feedback is not just helpful—it is a necessity.

It is also important to understand that **feedback is not the same as criticism**. Criticism often focuses on what went wrong, sometimes in a negative or judgmental way. It can make people feel attacked or discouraged, especially if it targets the person rather than the behavior.

Feedback, on the other hand, is meant to be constructive. It aims to help someone improve by offering specific, actionable insights. While criticism tends to close people down, feedback opens the door to learning. The intention behind feedback is growth, not blame. This difference in intention is what makes feedback a powerful tool rather than a harmful one.

To ensure feedback is effective, there are several **ground rules** that should always be followed. First, feedback should be **specific**. Vague comments like "You need to do better" are not helpful; instead, the feedback should clearly describe what happened and what could be improved. Second, feedback should be **timely**. Giving feedback too long after the event makes it less relevant and harder to apply. Third, feedback should be **focused on behavior, not personality**. This means addressing what someone did, not who they are. Fourth, feedback should be **balanced**. It is important to acknowledge what went well as well as what needs improvement. Finally, feedback should be **respectful and supportive**, creating a safe environment where the receiver feels encouraged rather than judged.

An example of effective feedback I received recently came from a colleague during a group project. I had taken the lead on organizing the tasks, but in doing so, I unintentionally made some decisions without consulting the rest of the team. Instead of criticizing me, my colleague approached me privately and said, "I appreciate how you took initiative and kept the project moving, but sometimes I feel left out of the decision-making. It would help me if we could discuss the next steps together before finalizing them." This feedback was specific, respectful, and focused on behavior rather than personality. It helped me understand how my actions affected others, and it strengthened our teamwork. I adjusted my approach, and the project became more collaborative and successful as a result.

In conclusion, feedback is an essential tool for growth, learning, and effective communication. It supports key 21st-century skills, strengthens relationships, and helps individuals and teams perform at their best. When delivered properly—respectfully, specifically, and constructively—feedback becomes a powerful force for positive change.

Week 8 Planning & Organising:

Planning & Organising
Assignment Week 8, Period 3 (30-03/03-04)

Name: Slman Al-Odani
Date: 15/04/2026

1. Download this document and answer the questions here under
2. When finished, upload your assignment on Blackboard, in your Portfolio
3. **Deadline: 19-04, at 23:59**

This week's assignment is simple. Write a 500 words reflection on your Experience with Collaboration so far. Include the following elements in your essay:

Looking back

1. What happened exactly?
2. What went well and what went less well?
3. Why did I approach it like this?
4. What do I learn about myself?

Looking forward

1. When did I succeed in the past?
2. What will I do differently next time?
3. What will I do the same next time?

Your Reflection:

Looking back at the past weeks of working on my project, I can see that collaboration has played a much bigger role than I expected. At first, I thought building my Flask application would be a mostly individual task, where I just write code and fix bugs on my own.
But I ended up collaborating in different ways: asking questions, getting feedback, comparing solutions with classmates, and even working together with tools like Copilot to understand concepts I didn't fully get at first.

What happened exactly?
I spent a lot of time building the experiment-tracking system, and along the way I kept running into parts I didn't fully understand—like how Blueprints work, how to structure routes, and how to calculate things like averages and cutoff dates. Instead of struggling alone, I reached out for help. Sometimes that meant asking classmates how they approached something, and sometimes it meant asking Copilot to explain a piece of code or rewrite it in a clearer way. This back-and-forth became a form of collaboration that helped me move forward faster.

What went well and what went less well?

What went well was communication. Whenever I asked for help, I got clear explanations that made sense to me. I also realized that explaining my own code to others helped me understand it better. What went less well was that sometimes I waited too long before asking for help. I would get stuck on something small, like a date calculation or a list comprehension, and instead of reaching out immediately, I tried to solve it alone for too long. That slowed me down more than it helped.

Why did I approach it like this?
I think part of me wanted to prove that I could do it independently. As a first-year CS student, it's easy to feel like asking for help means you don't understand enough. But the more I collaborated, the more I realized that everyone asks questions, and that collaboration is part of the learning process, not a sign of weakness.

What do I learn about myself?
I learned that I work better when I talk through problems instead of keeping them in my head. I also learned that I understand concepts much faster when someone explains them in simple terms. Collaboration makes me more confident, not less.

Looking forward, I can see moments where I succeeded in the past—especially when I asked for help early, shared my progress, or compared solutions with others. Those moments always pushed the project forward.

What will I do differently next time?
Next time, I want to ask for help sooner instead of wasting time being stuck. I also want to share my code earlier so I can get feedback before problems grow bigger.

What will I do the same next time?
I will keep collaborating, keep asking questions, and keep breaking down problems with others. That approach clearly works for me, and it makes the whole process less stressful and more enjoyable.

Period 4:

Week 4 360 DEGREE FEEDBACK:

360 DEGREE FEEDBACK FORM

FROM: Hamood (please fill your name)

TO: Slman (please fill the name of the person you are giving feedback to)

RELATION TO FEEDBACK ASKER: Father

ME

How would you describe me?

Your answer: Serious, smart, and very driven. You know what you want and you go after it.

What do you see as my strength(s)?

Your answer: Your focus and commitment. Once you set your mind on something, nothing stops you.

Can you describe a situation when I demonstrated this strength?

Your answer: You stayed home to study even when we invited you to come out with the family. That kind of discipline is not easy to find.

ME IN A TEAM

How do you see my role in this team?

Your answer: -----

What is my contribution to this team?

Your answer: -----

What would you ask me to help you with?

Your answer: -----

TIPS

Do you see any room for improvement for me as a team member?

Your answer: Learn to step back sometimes — work with people, not just beside them. Balance matters.

Do you see any room for improvement for me as a future IT-specialist?

Your answer: Go outside more, talk to people, get some fresh air. A rested mind works better than an exhausted one.

What competences would you recommend me to work on?

Your answer: Work-life balance.

360 DEGREE FEEDBACK FORM

FROM: Nagat (please fill your name)

TO: Slman (please fill the name of the person you are giving feedback to)

RELATION TO FEEDBACK ASKER: Mother

ME

How would you describe me?

Your answer: Hardworking and dedicated. You never give up on what you start.

What do you see as my strength(s)?

Your answer: Your strong work ethic and ability to stay focused for long hours.

Can you describe a situation when I demonstrated this strength?

Your answer: You stayed up late finishing your CS assignments even when it was hard. You never stopped until everything was done properly.

ME IN A TEAM

How do you see my role in this team?

Your answer: -----

What is my contribution to this team?

Your answer: -----

What would you ask me to help you with?

Your answer: -----

TIPS

Do you see any room for improvement for me as a team member?

Your answer: Don't do everything alone — share your work with others and ask for help when you need it.

Do you see any room for improvement for me as a future IT-specialist?

Your answer: Keep pushing, but also take care of yourself. You can't give your best when you're exhausted.

What competences would you recommend me to work on?

Your answer: Time management and self-care.

360 DEGREE FEEDBACK FORM

FROM: Sally (please fill your name)

TO: Slman (please fill the name of the person you are giving feedback to

RELATION TO FEEDBACK ASKER: Sister

ME

How would you describe me?

Your answer: Responsible and respectful.

What do you see as my strength(s)?

Your answer: Creativity.

Can you describe a situation when I demonstrated this strength?

Your answer: You suggested good ideas when we wanted to surprise our sister for her birthday.

ME IN A TEAM

How do you see my role in this team?

Your answer: -------

What is my contribution to this team?

Your answer: ------

What would you ask me to help you with?

Your answer: ----

TIPS

Do you see any room for improvement for me as a team member?

Your answer: Try to participate more in meetings.

Do you see any room for improvement for me as a future IT-specialist?

Your answer: Improve your confidence.

What competences would you recommend me to work on?

Your answer: Communication.

360 DEGREE FEEDBACK FORM

FROM: Sjonly (please fill your name)

TO: Salmon (please fill the name of the person you are giving feedback to

RELATION TO FEEDBACK ASKER: Team Member

ME

How would you describe me?

Your answer: dedicated soldier

What do you see as my strength(s)?

Your answer:
Your dedication to always reach class on time and always finish your assignments on time without showing any signs of tiredness

Can you describe a situation when I demonstrated this strength?

Your answer: when I would come to school tired and I see you there early and working on your assignments.

ME IN A TEAM

How do you see my role in this team?

Your answer: great team members always do your part and still help me understand my parts.

What is my contribution to this team?

Your answer: motivation, inspiration and you're like a backbone, very needed.

What would you ask me to help you with?

Your answer: time management

TIPS

Do you see any room for improvement for me as a team member?

Your answer: No, you're already good.

Do you see any room for improvement for me as a future IT-specialist?

Your answer: everyone needs improvement

What competences would you recommend me to work on?

Your answer: you don't have any

360 DEGREE FEEDBACK FORM

FROM: Mariam (please fill your name)

TO: Slman (please fill the name of the person you are giving feedback to

RELATION TO FEEDBACK ASKER: Sister

ME

How would you describe me?

Your answer: Smart and cooperative.

What do you see as my strength(s)?

Your answer: Fast learner.

Can you describe a situation when I demonstrated this strength?

Your answer: You quickly understood the task I gave you regarding some math problems.

ME IN A TEAM

How do you see my role in this team?

Your answer: ----------

What is my contribution to this team?

Your answer: ----------

What would you ask me to help you with?

Your answer: ------

TIPS

Do you see any room for improvement for me as a team member?

Your answer: Maybe improve confidence in presentations.

Do you see any room for improvement for me as a future IT-specialist?

Your answer: Gain more experience with technical projects.

What competences would you recommend me to work on?

Your answer: leadership skills.

360 DEGREE FEEDBACK FORM

FROM: Soliman (please fill your name)

TO: Slman (please fill the name of the person you are giving feedback to

RELATION TO FEEDBACK ASKER: Brother

ME

How would you describe me?

Your answer: Friendly, calm, and hardworking.

What do you see as my strength(s)?

Your answer:
You listen well and work seriously.

Can you describe a situation when I demonstrated this strength?

Your answer: When you help me with my homeworks.

ME IN A TEAM

How do you see my role in this team?

Your answer: ----------

What is my contribution to this team?

Your answer: ----------

What would you ask me to help you with?

Your answer: ----------

TIPS

Do you see any room for improvement for me as a team member?

Your answer: Try to share your opinions more often.

Do you see any room for improvement for me as a future IT-specialist?

Your answer: Practice technical skills more regularly.

What competences would you recommend me to work on?

Your answer: Coding skills.

Week 8 Feedforward, Feed Up and More:

Feedforward, Feed Up and More
Assignment Week 6, Period 4 (01-06/05-06)

Name Slman Al-Odani
Date 7/6/2026

1. Download this document and answer the questions here under
2. Sit down with your recently formed team of 3-4 students and do assignment nr 1
3. Do the other assignment in this document individually
4. When finished, upload your assignment on Blackboard, in your Portfolio
5. **Deadline: 07-06, at 23:59**

Assignment 1 (group assignment) During Theory class

You get this poorly defined assignment : "Write a 1500 words essay about Professional Skills". Rewrite this assignment using feed up. What (criteria) would you add/ask for to the description of this assignment, to hand in the best essay ever?
Write a 1500-word essay in which you critically analyze what professional skills are and why they are essential in your future career. Your essay must define professional skills using at least one academic source, identify and argue which three skills are most relevant to your field, and reflect personally on your own development. Use a clear essay structure (introduction, body, conclusion), write in full paragraphs and no bullet points.

Assignment 2 (pair assignment) During Process class

Your pair explains a recent challenge (study, presentation, teamwork)
You give **only feedforward.** So: no judgment & at least 2 concrete suggestions.
Assignment:
1. Write down here in 1 sentence what your pair's challenge is.
2. Write down here minimum 2 concrete suggestions (your feedforward to your pair) you gave
1- He struggled to speak up during a group presentation because every time he had a point to add, someone else jumped in first and he ended up just standing there while the rest of the group carried it.
2 - Claim a slide before the presentation starts and use a short physical signal to take the floor.

Assignment 3 (individual assignment) In your own time – Reflection and Transfer

Assignment: write a small essay (max 250 words) on the following:

1. Where do I need more feed up in my studies?
2. How can I ask for feedforward effectively?
3. How will I use feedforward in group work?

Theory-heavy courses are where I need the most feed up. I often follow a lecture in the moment but get home and realize I have no idea what I was actually supposed to take from it — memorize it, analyze it, link it to practice? I tell myself I will figure it out later, but that rarely happens. What I need upfront is a clear goal: what does this topic ask of me, and what does a good answer look like?

When I do not understand something I usually Google it, which helps with facts but not with direction. Asking a teacher feels awkward, so I avoid it. A better approach is to write down one specific question during class — not "I don't get it" but something like "are we expected to apply this or just explain it?" That gives the teacher something concrete to respond to, and I can ask after class or by email without it feeling like a big deal.

My contribution in groups depends on whether I have a clear role — without one I tend to wait and take whatever is left. Going forward I want to ask at the start of every project what each person needs from the others. If I know exactly what is expected I will work better, and most teammates probably feel the same way.

Semester 2 reflection:

Semester 2 felt different from the start. We moved into territory that actually challenged me to think differently — not just about coding, but about what technology can do and how it works underneath. The main focus was AI: we went through CS50's Introduction to AI, which covered everything from search algorithms to machine learning, and we were expected to apply that knowledge in two real projects. That combination of theory and practice made this semester feel more serious than the last one.

The solo project, Solo Innovate, was something I had already started building, but this semester I had to take it further by integrating AI into it. That pushed me to actually understand what I was implementing rather than just copying something that worked. It forced me to connect what I was learning in CS50 with something I had already built, which honestly made the concepts stick better than any assignment would have.

The group project, Checkmate, was an attendance monitoring system built on a Raspberry Pi 4 using Python, Flask, and hardware modules. On paper it sounds straightforward, but in practice it was a completely different story. Each person owned a specific part of the system — BLE scanning, NFC, the database, the API, the Flask dashboard — and everyone built their module mostly on their own. The hardest part came when we had to bring it all together. Different people had written their code in different ways, with different assumptions about how the other parts would behave, and making everything talk to each other without breaking anything was genuinely hard. You don't realize how much can go wrong at the seams until you're actually sitting there trying to connect different modules into one working system.

What I can say is that I held up my end. Every task the group assigned me, I got done. I wasn't the person who left things to the last minute or went quiet when deadlines were close. That matters to me, not because I want credit for it, but because I know group work only functions when everyone pulls their weight — and I didn't want to be the one who didn't.

As a group, we actually did well. Everyone contributed. Nobody disappeared. We did have that very familiar experience of finishing everything in the final week, which was stressful, but we got it working. Looking back, I think we could have started the merging process earlier instead of each person working in isolation for too long. That's the part I would change if we did it again.

Coming out of this semester, I feel ready for what's next. Not overconfident, but genuinely motivated. I understand AI better now — not just how to use tools that involve it, but some of the logic behind how it actually works. That feels like a real step forward. Next year I want to start earlier, communicate more during the building phase, and keep being the person who delivers when it counts.